

Medición y Métricas de Software

Medición del software

La medición del software permite obtener valores numéricos sobre atributos de componentes, sistemas o procesos de software. Comparar estos valores con estándares organizacionales facilita evaluar la calidad del software y la efectividad de herramientas y métodos. Un ejemplo es medir la cantidad de defectos descubiertos antes y después de implementar una nueva herramienta de pruebas para evaluar su efectividad.

El objetivo a largo plazo es reemplazar revisiones manuales por mediciones que determinen la calidad del software. Sin embargo, la valoración automatizada aún no es una realidad alcanzable.

Métricas de software

Una métrica de software es una característica medible objetivamente en un sistema, documentación o proceso de desarrollo. Ejemplos incluyen el tamaño del código en líneas, la legibilidad de un texto (índice Fog), el número de fallas reportadas o el esfuerzo en días-hombre para desarrollar un componente.

Existen dos tipos de métricas:

1. **Métricas de control:** sirven para gestionar procesos y evaluar su rendimiento (ej. tiempo para corregir defectos).
2. **Métricas de predicción:** estiman características del software (ej. complejidad ciclomática, longitud de identificadores en código).

Ambas métricas influyen en decisiones administrativas, ya sea para mejorar procesos o estimar el esfuerzo necesario en modificaciones del software.

Uso de las mediciones en software

Las mediciones pueden utilizarse para:

1. **Asignar valores a atributos de calidad** como mantenibilidad, agregando mediciones de componentes individuales.

2. **Identificar componentes deficientes** en comparación con estándares, por ejemplo, detectar módulos con alta complejidad que podrían contener más errores.

Dificultades en la medición de calidad

Los atributos de calidad como mantenibilidad, usabilidad y comprensibilidad son externos y subjetivos, lo que dificulta su medición directa. Para evaluar calidad, se mide atributos internos (ej. tamaño, complejidad) y se asume una relación con los atributos externos de interés.

Para que una métrica interna prediga un atributo externo de calidad, deben cumplirse tres condiciones:

1. **Medición exacta del atributo interno**, usando herramientas especializadas.
2. **Relación comprobable entre el atributo medido y el atributo de calidad externo**.
3. **Definición y validación de un modelo funcional** (ej. lineal o exponencial) basado en datos reales.

Las herramientas de análisis de código pueden medir atributos internos como la complejidad ciclomática. Sin embargo, aunque se supone que la complejidad se relaciona con la cantidad de fallas, demostrarlo requiere grandes volúmenes de datos, los cuales pocas empresas recopilan de forma consistente.

La medición del software es clave en la ingeniería de software empírica, que busca validar métodos y técnicas a través de datos reales. No obstante, la toma de decisiones en la industria aún se basa en factores subjetivos, como la novedad y el interés profesional, lo que retrasa la adopción de resultados empíricos.

Las métricas del producto miden atributos internos del software y se dividen en dos tipos:

1. **Métricas dinámicas**: recopiladas en tiempo de ejecución, útiles para evaluar eficiencia y fiabilidad (ejemplo: número de fallos).
2. **Métricas estáticas**: derivadas del código y documentación, útiles para valorar complejidad y mantenibilidad (ejemplo: tamaño del código).

Las métricas dinámicas tienen una relación clara con la calidad del software, mientras que las métricas estáticas presentan correlaciones indirectas. Entre las métricas más relevantes están:

- **Fan-in/Fan-out**: indican el grado de acoplamiento y la complejidad del control.
- **Longitud de código**: predice la propensión a errores.
- **Complejidad ciclomática**: relacionada con la comprensibilidad.
- **Longitud de identificadores**: afecta la claridad del código.
- **Profundidad de anidado condicional**: cuanto mayor es, más difícil es entender el código.

- **Índice Fog:** mide la dificultad de comprensión de documentos.

Estos parámetros ayudan a predecir y mejorar la calidad del software, aunque su efectividad depende del contexto de desarrollo y la tecnología utilizada.

Métricas orientadas a objetos

Las métricas orientadas a objetos (OO) surgieron en la década de 1990 y, a pesar de los avances en la industria del software, siguen siendo las más utilizadas. Herramientas de diseño UML pueden recopilar estas métricas automáticamente. Sin embargo, según El-Amam (2001), aún no existe suficiente evidencia para correlacionar estas métricas con la calidad del software, y esta situación se mantiene hasta la actualidad.

Algunas métricas clave en software orientado a objetos incluyen:

- **Métodos ponderados por clase (WMC):** mide la cantidad de métodos en una clase, ponderados por su complejidad. Valores altos sugieren clases más complejas y difíciles de reutilizar.
- **Profundidad de árbol de herencia (DIT):** indica la cantidad de niveles en la jerarquía de herencia. Un DIT alto puede hacer que el diseño sea más difícil de comprender.
- **Número de hijos (NOC):** representa la cantidad de subclases directas de una clase. Un valor alto puede indicar mayor reutilización, pero también mayor esfuerzo en validación.
- **Acoplamiento entre clases de objetos (CBO):** evalúa la dependencia entre clases. Un CBO elevado sugiere que modificaciones en una clase pueden afectar a otras.
- **Respuesta por clase (RFC):** número de métodos que pueden ejecutarse en respuesta a un mensaje recibido. Valores altos sugieren mayor complejidad y probabilidad de errores.
- **Falta de cohesión en métodos (LCOM):** mide la cohesión dentro de una clase, comparando métodos que comparten o no atributos. Existen múltiples variaciones de esta métrica, pero su utilidad sigue en debate.

Análisis de componentes de software

El análisis de componentes del software implica medir diferentes aspectos de los módulos y compararlos con datos históricos. Mediciones anómalas pueden indicar problemas de calidad en ciertos componentes.

Las etapas clave en este proceso incluyen:

1. **Elegir las mediciones a realizar:** se formulan preguntas clave y se define qué métricas recopilar.
2. **Seleccionar componentes a valorar:** no siempre es necesario medir todos los módulos, sino que puede seleccionarse una muestra representativa.

3. **Medir características de los componentes:** se usan herramientas automatizadas para calcular valores métricos.
4. **Identificar mediciones anómalas:** se comparan valores con datos históricos y se identifican valores inusuales.
5. **Analizar componentes anómalos:** se evalúan los módulos con valores anormales para determinar si reflejan problemas de calidad reales.

Se recomienda almacenar datos históricos de métricas para mejorar la evaluación de calidad del software en futuros proyectos y validar relaciones entre métricas internas y calidad externa.

Ambigüedad de mediciones

Las mediciones de software deben analizarse en contexto, ya que pueden ser malinterpretadas. Un ejemplo es la relación entre peticiones de cambio y la calidad del software:

- Se podría asumir que un alto número de peticiones de cambio indica baja calidad.
- Sin embargo, también podría significar que el software es popular y ampliamente utilizado.
- Cambios en los procesos pueden influir en la cantidad de peticiones sin necesariamente reflejar mejoras o deterioros en la calidad.

Para interpretar correctamente los datos, es necesario conocer quién realiza las peticiones, su motivo y el contexto del mercado. Los datos cuantitativos por sí solos no siempre reflejan la realidad y deben analizarse con precaución antes de extraer conclusiones definitivas.